

Exploring Lattice-based Post-Quantum Signature for JWT Authentication: Review and Case Study

Abdolmaged Alkhulaifi and El-Sayed M. El-Alfy

Department of Information and Computer Science, King Fahd University of Petroleum and Minerals, Saudi Arabia

Abstract—Due to the scalability of the stateless and compact JSON Web Tokens (JWT), they are increasingly used in securing modern applications to support single-sign-on context and establish trust in microservices or connected-devices architectures. However, currently JWT relies on traditional encryption algorithms that are based on integer factorization or discrete logarithm problems; this exposes JWT signatures to vulnerability when quantum computers become available. Recently, NIST has announced new standards which are quantum safe. In this paper, we investigate the usage of two quantum-resistant algorithms, which are NIST candidates of round 2 standardization process in 2019, to create and verify JWT signatures. Namely, we consider lattice-based schemes (DILITHIUM and qTESLA) and compare their performance against RSA for digital signature for different security levels defined by NIST. The results show that lattice-based digital signature schemes have better performance than RSA in terms of request per seconds and average response time. DILITHIUM has demonstrated the best performance while also having smaller performance loss when scaling the security level.

I. INTRODUCTION

Online user authentication is the process of identifying a user as being the claimed person. Once authenticated, the server can grant access to authorize the user according to his role or privileges. Common approaches are based on storing information in the sever side to be used for verification. However, checking the server database every-time the user makes a request would cause a serious performance bottleneck. Unlike cookies and session-based identifiers, token-based authentication is a stateless process designed to ease the security check without overloading the server. After the user logs in successfully, the server generates a token containing information that identifies the user and stores it in the client side. The obtained token will allow the user to fetch a specific resource without using his/her username and password by offering the token with each succeeding request. It will also help the server to quickly identify and trust the user without the need to access the database.

A summary of the main differences between session and token-based authentication is shown in Table I. Token-based authentication can be a prominent lightweight scalable solutions for decentralized environments such as web services, microservice applications, Internet-of-Things (IoT), and connected vehicles. Authentication in a multi-server system with session-based authentication can be very complicated. Also if session-based authentication is used, it can result in more overhead in the server side to store session information of all users and to check it every time a user makes a request [1].

TABLE I
SESSION-BASED VERSUS AND TOKEN-BASED AUTHENTICATION

Criteria	Session-based	Token-based
Client-side storage	Contains the user session ID	Contains the user signed and verified information
Database lookup	Requires a database to lookup session information every time the user makes a request	No database lookup is required
Client state	Requires sub-subsequent requests from the same user to hit the same server to preserve client state	Client's state is stored in the client side
Scalability	Scaling is difficult	Scalable

Being stateless is considered to be good for balancing server loads, which will contribute to its scalability [2]. Achieving scalability with statefull services is difficult as when the load balancer forward the user requests to another server, his session information need also either to be copied to the other server or to have a central server that stores the session for all the clients.

One of the most widely used token-based authentication format is JSON Web Token (JWT) [3] due to the flexibility of the JSON formatted data. JWT authentication has been employed in many different use cases such as healthcare services [4], Cloud SaaS (software as a service) access management [5], MQTT (message queuing telemetry transport) authentication for IoT devices [6], [7], etc. By using JWT, the server stores an access token that contains user information structured in JSON format and a digital signature. By verifying the signature, the server can detect any changes in the token information, e.g. an eavesdropper changes the ID field in the token to impersonate the identity of another user and gain access to his/her information in an online service.

The current widely-used algorithms for generating JWT signatures are HMAC, RSA or ECDSA with SHA256. However, public-key algorithms such as RSA are not quantum safe and recently the National Institute of Standards and Technology (NIST) is working on new standards in anticipation of the future of quantum computers. In this paper, we present a review and evaluate two lattice-based digital signature schemes that are second-round candidates in the context of JWT authentication.

The rest of the paper is organized as follows. Section II presents a brief background on token-based authentication and post-quantum cryptography. Section III presents a review of relevant related work. Section IV presents a case study to evaluate and compare two lattice-based schemes for JWT

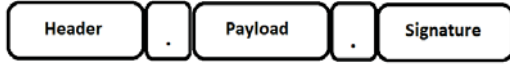


Fig. 1. JWT token structure

TABLE II
SIGNING ALGORITHMS FOR JWT AS DEFINED IN THE SPECIFICATION [8]

"alg" parameter	Digital Signature or MAC Algorithm	Implementation Requirements
HS256	HMAC using SHA-256	Required
HS384	HMAC using SHA-384	Optional
HS512	HMAC using SHA-512	Optional
RS256	RSASSA-PKCS1-v1_5 using SHA-256	Recommended
RS384	RSASSA-PKCS1-v1_5 using SHA-384	Optional
RS512	RSASSA-PKCS1-v1_5 using SHA-512	Optional
ES256	ECDSA using P-256 curve and SHA-256	Recommended
ES384	ECDSA using P-384 curve and SHA-384	Optional
ES512	ECDSA using P-521 curve and SHA-512	Optional
None	No digital signature or MAC performed	Required

signature generation and verification. Section V summarizes the results and concludes the paper.

II. BACKGROUND

A. JSON Web Token (JWT)

JWT token is a string that contains three parts separated by dots: header, payload and signature (as shown in Fig. 1). The header and payload contain data in JSON format and encoded in base64. Two common header fields are: token type which in this case JWT and algorithm used for signing/encrypting such as HMAC or RSA. The payload contains custom information like data about the client, token issue date, issuer information, etc. The signature holds the hash of the encoded header and encoded payload using the algorithm specified in the header. The hash is produced with a secret key and is used to verify the integrity of the content of the token. In case of using public key encryption, such as RSA, the hash is produced without a secret key but it's encrypted with the private key. This allows not only the main online service that issued the token, but also other services that is partnered with the issuer to decrypt the signature using the public key and extract the hash, which then is compared with the hash of the encoded header and payload to check if the client information has been tampered with. The cryptographic algorithms [8] defined in JWT specification for the signature are as shown in Table II.

The use of public-key encryption in JWT signature is essential in scenarios where an organization has multiple services. If a service 'A' wants to allow other services to identify and serve users associated with 'A', using the symmetric HMAC hashing for JWT signature requires 'A' to share its key with other services to allow them to verify the signatures of users of 'A'. Using this method will carry a huge security threat as it will allow services to generate a new valid JWT that belongs to any user using the same key. By using a public-key encryption, service 'A' will produce a hash of header and payload without a secret key and encrypt it with its private key. To allow users of 'A' to be authenticated in other services, 'A' would share

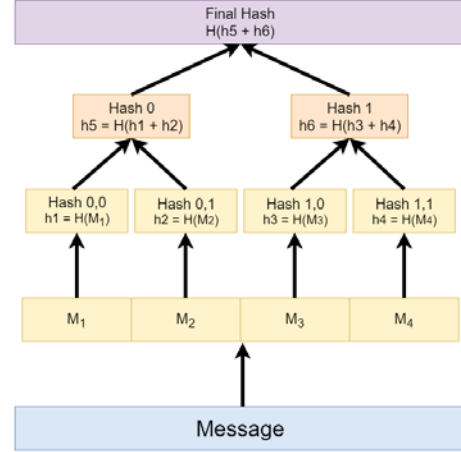


Fig. 2. Example of Merkle hash tree.

its public key to the other services. As can be noted from Table II, JWT signature asymmetric encryption algorithms use either RSA or ECDSA. Both algorithms rely on integer factorization problem or elliptic-curve discrete logarithm problem, respectively, which is vulnerable to attacks from large quantum computers [9]. The threat of quantum computers will expose all services that preform authentication with JWT token using either RSA or ECDSA, to impersonation and identity theft.

B. Post-Quantum Cryptography

Most currently used cryptography, such as RSA and ECDSA, relies on the hardness of integer factorization or elliptic-curve discrete logarithm problem. However, Shor [10] introduced a quantum algorithm which can solve integer factorization in polynomial time, which in turn would break RSA encryption. Furthermore, in [11] it is shown that Shor's algorithm can be used to break elliptic-curve-based cryptography. These quantum algorithms would render current cryptosystems insecure in the future where large quantum computers are built. While most current symmetric-cryptographic algorithms and hash functions are considered to be relatively secure against attacks by quantum computers, Groover [12] introduced a quantum algorithm that speeds up attacks against symmetric ciphers. However, doubling the key size can effectively block these attacks [13]. A new era in cryptography has emerged called post-quantum cryptography proposing new cryptography techniques (usually public-key algorithms) that can resist quantum computer attacks. The research on post-quantum cryptography is focused on five different directions [14]:

- Multivariate cryptography
- Hash-based cryptography
- Code-based cryptography
- Supersingular elliptic curve isogeny cryptography
- Lattice-based Cryptography

Most post-quantum cryptographic systems proposed to NIST belong to lattice-based or multivariate-based cryptography. In this paper, we focus on lattice-based cryptography

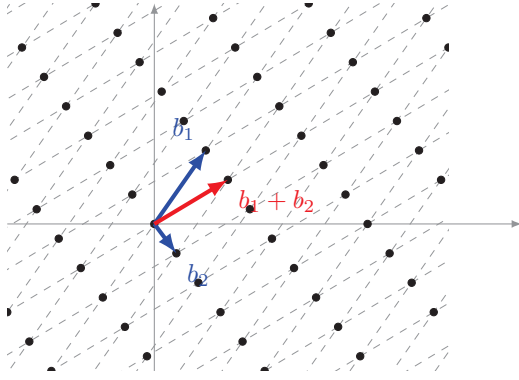


Fig. 3. Example of a lattice in $\mathbb{R}^n$ with two basis

due to the algorithm used being relatively simple, efficient and highly parallelizable while also some lattice-based systems are provably secure under a worst-case hardness assumption, rather than on the average case [15].

III. RELATED WORK

The first lattice-based cryptosystem was introduced in 1996 by Ajtai [16]; its security was based on the hardness of the shortest vector problem. However, this cryptosystem was a symmetric-key cryptography with worst-case notion. In 1998, a new public-key encryption scheme is proposed based on the hardness of the shortest vector problem [17]. Their new scheme is called NTRU. However, this scheme was not proven to be at least as hard as solving a worst-case lattice problem. It was later shown to be vulnerable to two type of attacks: in 2003 Graham et al. [18] showed that it's vulnerable to the meet-in-the-middle attack and in 2000 Jaulmes et al. [19] showed that it is vulnerable to chosen-ciphertext attack.

The first cryptosystem scheme that is lattice-based and has its security proven to be under worst-case hardness assumptions of lattice problems was introduced in 2005 by Regev [20]. Regev was also the first one to introduce the learning with error problem. In 2009 [21], an improved version of Regev learning with error lattice-based cryptosystem is proposed by assuming the worst-case hardness of approximating the minimum distance. In 2003, Brakerski et al. [22] proposed another modified version of Regev cryptosystem by employing extended learning with errors. Many research was later conducted to improve Regev cryptosystem performance. In 2010, Lyubashevsky et al. [23] proposed to use an algebraic variant of learning with error called ring-learning with error that is more efficient than the original learning with error. In 2014, Peikert [24] proposed a more optimized variant of the ring learning with error that has a reduced length in ciphertext. These mentioned cryptosystem are not candidates to be evaluated and standardized as quantum-resistant public-key cryptographic algorithms by NIST.

A candidate of NIST post-quantum lattice-based standards for digital signature is DILITHIUM [25], its hardness is based on module learning with error and the short integer solution problem. The procedure for generating the key, signing and

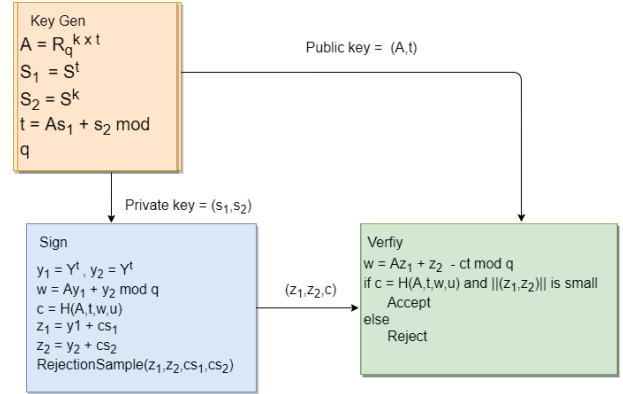


Fig. 4. DILITHIUM scheme for generating keys, signing and verifying signatures

verifying signature are viewed in Fig. 4. DILITHIUM design was based on [26] and [27] but without Gaussian sampling. Also another improvement of DILITHIUM over [26] and [27] is that the public key size is shrunk by more than a factor of 2. Another candidate is qTESLA which was proposed by Alkim et al. [28] based on the ring learning with errors and self-target ring short integer solution problem. The design of qTESLA is based on the Bai-Galbraith scheme [27]. The main feature of the design of qTESLA was to be a direct replacement to the existing X.509 certificate.

In the context of performance evaluation of JWT, Rahmatulloh et al. [29] compared the performance of using HMAC, RSA and ECDSA with SHA256. Their test consisted of measuring token generation time, token size and token transfer time. The results from their experiment showed that HMAC generation and transfer time were much lower than RSA or ECDSA. But comparing the results of public-key signing-only, ECDSA outperforms RSA in generating and transferring tokens. A general performance comparison of RSA and ECDSA outside the context of JWT is demonstrated in [30], where ECDSA achieved better performance in key generation and is only faster than RSA for signature when keys are large. However, RSA performed much better than ECDSA in signature verification. A summary of the results of the study in [30] is shown in Table III, which contradicts the finding of [29], but it could be justified that the context of the experiment wasn't signing and verifying JWT tokens.

IV. EVALUATION

In order to evaluate the performance of lattice-based post-quantum digital signature schemes for JWT authentication, we have conducted several experiments and reported the results for signature generation and verification. We focused on post-quantum schemes of second-round candidates of NIST standardization for digital signature, namely DILITHIUM and qTESLA. We benchmarked the results to the traditional RSA digital signature scheme, which is currently commonly used in JWT-based authentication.

TABLE III
SUMMARY OF PERFORMANCE COMPARISON BETWEEN RSA AND ECDSA

Criteria	RSA					ECDSA				
Key Length (bits)	1024	2240	3072	7680	15360	163	233	283	409	571
Key Generation (sec)	0.16	7.47	9.80	133.90	679.06	0.08	0.18	0.27	0.64	1.44
Signing (sec)	0.01	0.15	0.21	1.53	9.20	0.15	0.34	0.59	1.18	3.07
Verifying (sec)	0.01	0.01	0.01	0.01	0.03	0.23	0.51	0.86	1.80	4.53

TABLE IV
COMPLIANCE TO NIST SECURITY LEVELS #1 AND #3; SIZES IN BYTES.

Criteria	RSA		CRYSTALS		qTESLA	
NIST security level	1	3	1	3	1	3
Public key length	384	960	1184	1760	14880	38432
Signature length	384	960	2044	3366	2592	5664

A Client-Server application was developed in Python. The server has two functions: (1) accepts an HTTP request and generates a new JWT that includes random data, or (2) accepts an HTTP request that contains JWT and verifies it. The JWT token body was first encoded to base64 format then hashed with SHA-256 and signed using the selected digital signature scheme and the resulting signature is then encoded into base64 and appended with the base64 of the JWT body creating the final JWT token. For signature verification, after base64 decoding the JWT body is hashed with SHA-256 then compared to the decrypted signature. The Python server has three variations depending on the adopted digital signature scheme (DILITHIUM, qTESLA, and RSA). We adopted an open-source C library called liboqs [31], which includes a collection of regularly updated quantum-safe cryptographic algorithms and conducted all experiments in Python using a Python wrapper for liboqs. For RSA, we used the RSASSA-PKCS1-v1.5 implementation by PyCryptoDome Python library which is the digital signature scheme used by JWT according to its specification [8].

The server was deployed in a VMware Workstation 14 running Ubuntu 18 with 4 cores Intel i7 4790k and 4GB DDR3 RAM. The use of virtual environment helps in isolating the server from any other running applications in the background while controlling the resources being used. Once the server is deployed, we performed HTTP stress test and recorded the requests per second and the average response time in signing and verifying JWT tokens.

The results are recorded for the parameters recommended by NIST security levels (#1 and #3). These levels compare the strength to breaking symmetric block cipher with key search attack. Security level #1 requires that any attack targeting the defined cipher scheme to have computational resources comparable of performing key search on AES-128. On contrast, NIST security level #3 requires the computational resources to be comparable of performing key search on AES-192 to break the cryptosystem of interest. The lengths of public key and signatures for each of the digital signature schemes according to different NIST security levels are shown in Table IV.

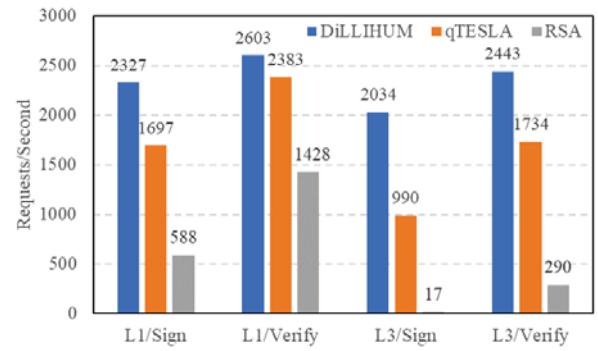


Fig. 5. Comparing algorithms based on requests per second

Fig. 5 shows the number of requests per second for each of the digital signature schemes with parameters corresponding to NIST security level #1 and #3. We can notice that lattice-based schemes have better performance than RSA in signing and verifying JWT signatures. For NIST level #1, DILITHIUM achieved the highest results with 2327 and 2603 requests per second for signing and verifying signatures, respectively, which makes its throughput 4 times better than RSA in signing and 1.8 times better than RSA in verifying. Next comes qTESLA with 1697 and 2383 requests per second in signing and verifying, respectively. Finally, RSA with a key size of 3072 bits (384 bytes), which is the equivalent of the same security level as its counterparts, achieved only 588 and 1428 requests per second in signing and verifying JWT token, respectively. Similar relative behavior can be observed for NIST level #3.

Fig. 6 shows the average response time in milliseconds for each of the digital signature schemes. We can see the same pattern of the dominance of lattice-based digital signature schemes. DILITHIUM is again ahead; with average response time for NIST level #1 of 21.5ms and 18.7ms in signing and verifying, respectively, followed by qTESLA, which has 29.4ms and 20.9ms in signing and verifying, respectively. RSA is again falling behind achieving 84.4ms and 34.9ms in signing and verifying. Compared to DILITHIUM, RSA is around 292% and 86% times slower in signing and verifying, respectively. Superiority of DILITHIUM is also observed for NIST level #3.

This performance gains comes at the cost of increased in bandwidth as we can notice from Table IV that lattice-based digital signature schemes require larger key size and

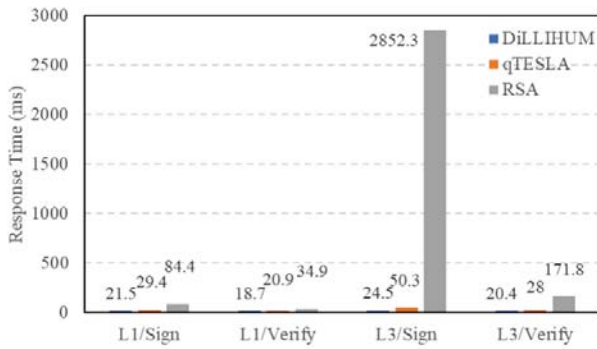


Fig. 6. Comparing algorithms based on average response time in milliseconds

produces larger signature size compared to RSA. This increase in signature size can be costly when hosting a server and paying per incoming and outgoing bandwidth used, as this increase will be applied to every API request since JWT will be always passed for authentication. However, when we compare DILITHIUM to qTESLA in term of signature size, DILITHIUM has smaller sizes of signature and public key.

V. CONCLUSION

This paper presented a brief review and surveyed related work on user authentication using JWT and Post-Quantum Cryptography. Moreover, the results of a case study are presented to evaluate the performance of utilizing two NIST candidates of lattice-based Quantum-Safe Cryptosystems for digital signature replacing the traditional forms of RSA and ECDSA in JWT authentication and authorization protocols. The results demonstrated performance superiority of DILITHIUM in throughput and response time compared to qTESLA and RSA. Concerns were also raised regarding the big increase in signature size of both DILITHIUM and qTESLA compared to RSA, which will result in a noticeable increase in bandwidth of the server. Moreover, DILITHIUM scales well with the increased security level with minimum performance loss. As future work, we are comparing more quantum-safe approaches.

ACKNOWLEDGMENT

The authors would like to thank King Fahd University of Petroleum & Minerals for support during this work.

REFERENCES

- [1] M. H. Barkadehi, M. Nilashi, O. Ibrahim, A. Z. Fardi, and S. Samad, "Authentication systems: A literature review and classification," *Teleatics and Informatics*, vol. 35, no. 5, pp. 1491–1511, 2018.
- [2] S. Lee, J.-Y. Jo, and Y. Kim, "Authentication system for stateless restful web service," *Journal of Computational Methods in Sciences and Engineering*, vol. 17, pp. 1–14, 11 2016.
- [3] M. Jones, "The emerging json-based identity protocol suite," *Information Standards Quarterly*, vol. 26, 11 2015.
- [4] P. Solapurkar, "Building secure healthcare services using oauth 2.0 and json web token in iot cloud scenario," in *Int. Conf. on Contemporary Computing and Informatics (IC3I)*, 2016.
- [5] O. Ethelbert, F. F. Moghaddam, P. Wieder, and R. Yahyapour, "A json token-based authentication and access management schema for cloud saas applications," in *IEEE Int. Conf. Future Internet of Things and Cloud (FiCloud)*, 2017.
- [6] A. Bhawiyuga, M. Data, and A. Warda, "Architectural design of token based authentication of mqtt protocol in constrained iot device," in *Int. Conf. Telecommun. Systems Services and Applications (TSSA)*, 2017.
- [7] K. Shingala, "Json web token (jwt) based client authentication in message queuing telemetry transport (mqtt)," *arXiv preprint arXiv:1903.02895*, 2019.
- [8] M. B. Jones, "Json web algorithms (jwa)," Internet Engineering Task Force (IETF), Standard RFC7518, 2015. [Online]. Available: <https://www.hjp.at/doc/rfc/rfc7518.html>
- [9] V. Mavroeidis, K. Vishi, M. D. Zych, and A. Jøsang, "The impact of quantum computing on present cryptography," *CoRR*, 2018.
- [10] P. W. Shor, "Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer," *SIAM Journal on Computing*, vol. 26, no. 5, pp. 1484–1509, Oct. 1997.
- [11] J. Proos and C. Zalka, "Shor's discrete logarithm quantum algorithm for elliptic curves," *Quantum Information & Computation*, vol. 3, no. 4, 2003.
- [12] L. K. Grover, "A fast quantum mechanical algorithm for database search," *arXiv preprint quant-ph/9605043*, 1996.
- [13] D. J. Bernstein, "Grover vs. mceliece," in *International Workshop on Post-Quantum Cryptography*, 2010, pp. 73–80.
- [14] —, "Introduction to post-quantum cryptography," in *Post-quantum cryptography*, 2009, pp. 1–14.
- [15] L. Chen, L. Chen, S. Jordan, Y.-K. Liu, D. Moody, R. Peralta, R. Perlner, and D. Smith-Tone, *Report on post-quantum cryptography*. US Department of Commerce, National Institute of Standards and Technology, 2016.
- [16] M. Ajtai, "Generating hard instances of lattice problems," in *Proceedings of the 28th Annual ACM Symposium on Theory of Computing*, 1996, pp. 99–108.
- [17] J. Hoffstein, J. Pipher, and J. H. Silverman, "Ntru: A ring-based public key cryptosystem," in *International Algorithmic Number Theory Symposium*, 1998, pp. 267–288.
- [18] N. Howgrave-Graham, J. H. Silverman, and W. Whyte, "A meet-in-the-middle attack on an ntru private key," Technical report, NTRU Cryptosystems, June 2003. Report, Tech. Rep., 2003.
- [19] É. Jaulmes and A. Joux, "A chosen-ciphertext attack against ntru," in *Annual International Cryptology Conference*, 2000, pp. 20–35.
- [20] O. Regev, "On lattices, learning with errors, random linear codes, and cryptography," *Journal of the ACM (JACM)*, vol. 56, no. 6, p. 34, 2009.
- [21] C. Peikert, "Public-key cryptosystems from the worst-case shortest vector problem," in *Proceedings of the 41st Annual ACM Symposium on Theory of Computing*, 2009, pp. 333–342.
- [22] Z. Brakerski, A. Langlois, C. Peikert, O. Regev, and D. Stehlé, "Classical hardness of learning with errors," in *Proceedings of the 45th Annual ACM Symposium on Theory of Computing*, 2013, pp. 575–584.
- [23] V. Lyubashevsky, C. Peikert, and O. Regev, "On ideal lattices and learning with errors over rings," in *Annual Int. Conf. Theory and Applications of Cryptographic Techniques*, 2010, pp. 1–23.
- [24] C. Peikert, "Lattice cryptography for the internet," in *Int. Workshop on Post-quantum Cryptography*, 2014.
- [25] L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, P. Schwabe, G. Seiler, and D. Stehlé, "Crystals-dilithium: A lattice-based digital signature scheme," *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2018, no. 1, pp. 238–268, Feb. 2018.
- [26] T. Güneysu, V. Lyubashevsky, and T. Pöppelmann, "Practical lattice-based cryptography: A signature scheme for embedded systems," in *International Workshop on Cryptographic Hardware and Embedded Systems*. Springer, 2012, pp. 530–547.
- [27] S. Bai and S. D. Galbraith, "An improved compression technique for signatures based on learning with errors," in *Cryptographers' Track at the RSA Conference*. Springer, 2014, pp. 28–47.
- [28] E. Alkim, P. S. Barreto, N. Bindel, P. Longa, and J. E. Ricardini, "The lattice-based digital signature scheme qtesla," *IACR Cryptology ePrint Archive*, vol. 2019, p. 85, 2019.
- [29] A. Rahmatulloh, R. Gunawan, and F. M. S. Nursuwars, "Performance comparison of signed algorithms on JSON web token," *IOP Conference Series: Materials Science and Engineering*, vol. 550, aug 2019.
- [30] N. Jansma and B. Arrendondo, "Performance comparison of elliptic curve and rsa digital signatures," *nicj. net/files*, 2004.
- [31] D. Stebila and M. Mosca, "Post-quantum key exchange for the internet and the open quantum safe project," in *Selected Areas in Cryptography – SAC 2016*, R. Avanzi and H. Heys, Eds., 2017, pp. 14–37.